Practical RTL Design

ASIC Design, Lecture 1

Overview of Topics

- Synthesis (RTL-to-Gate and Gate-to-RTL)
- Inferred Modules
- Critical Paths, Setup & Hold Violations
- PPA Metrics
- Finite State Machines
- Handshaking
- Metastability & CDC
- Synchronization & MTBF
- Synchronous FIFO Design

Big Picture

"I want a chip that can do A, B, and C"

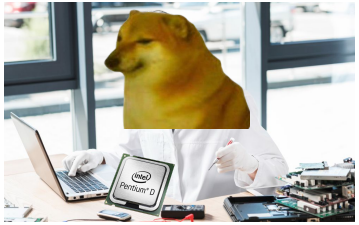




Chip Designer



Chip Foundry



Bringup & Validation Engineer



Consumer

Big Picture



Chip Designer

Design Specification

Architectural Design

Digital Design

Functional Verification

RTL Description (HDL)

Frontend Design

Synthesis

Logic Equivalence Check

Gate Level Netlist

DFT

Floorplanning

Place and Route

Physical Verification & Signoff Checks

Tapeout

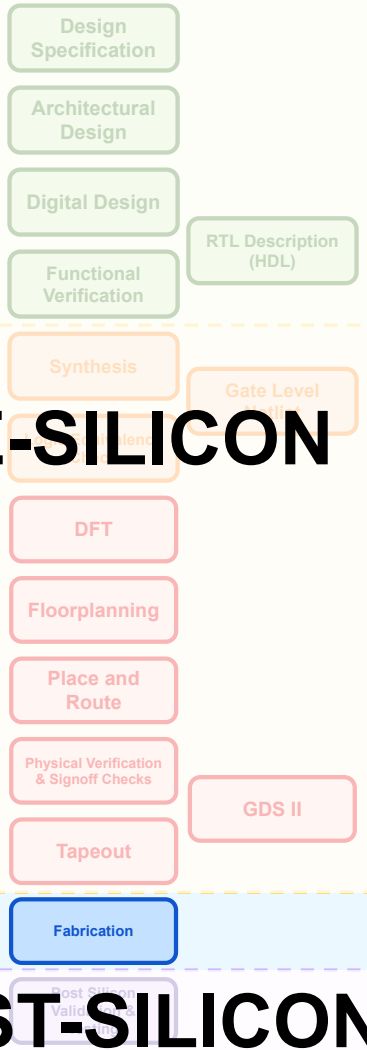
GDS II

Backend Design

Fabrication

Post Silicon Validation & Testing

Big Picture



Frontend Design

Backend Design

PRE-SILICON

POST-SILICON

Design Specification

Architectural Design

Digital Design

RTL Description (HDL)

Functional Verification

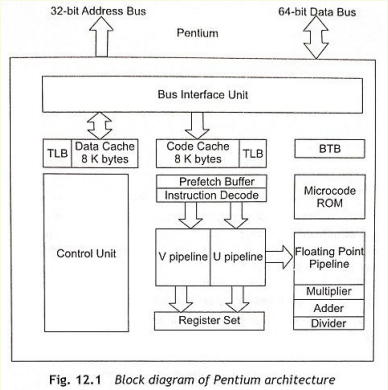


Fig. 12.1 *Block diagram of Pentium architecture*

```
module test (input clk, din, c1, c2, c3,
             output reg dout);

  always @(posedge clk) begin
    if (c1 & c2)
      if (~c2 & c3)
        dout <= din;
  end

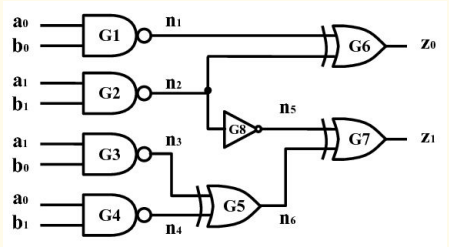
endmodule
```



Synthesis

Logic Equivalence Check

Gate Level Netlist



DFT

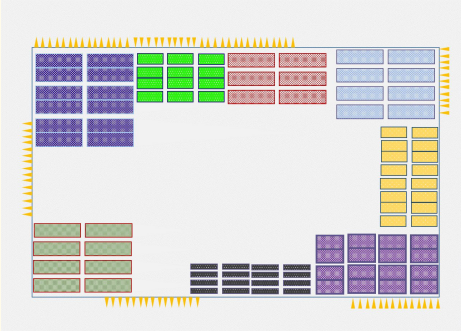
Floorplanning

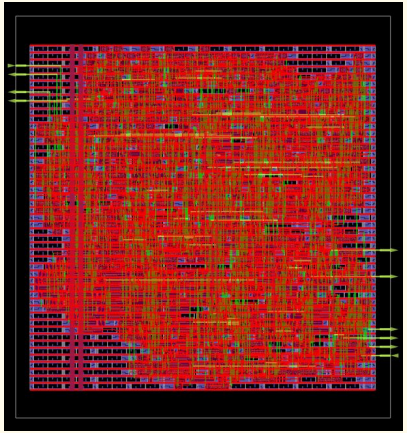
Place and Route

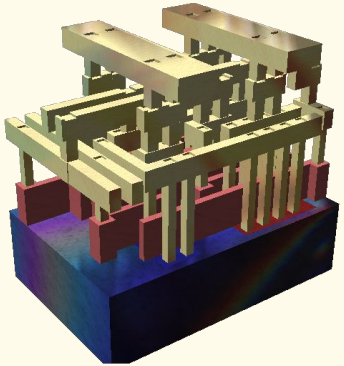
Physical Verification & Signoff Checks

Tapeout

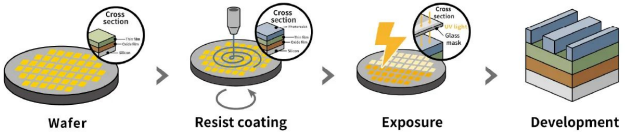
GDS II

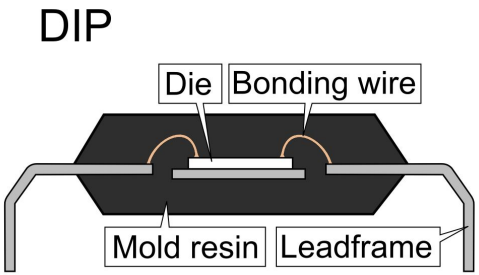




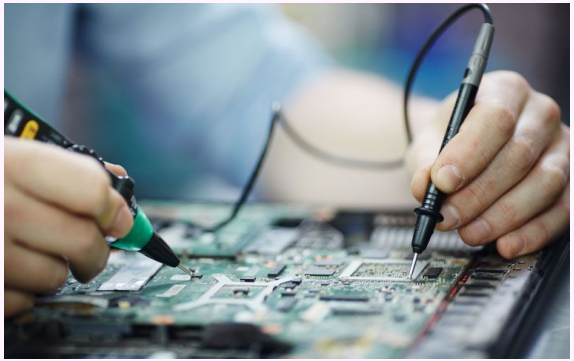


Fabrication





Post Silicon Validation & Testing

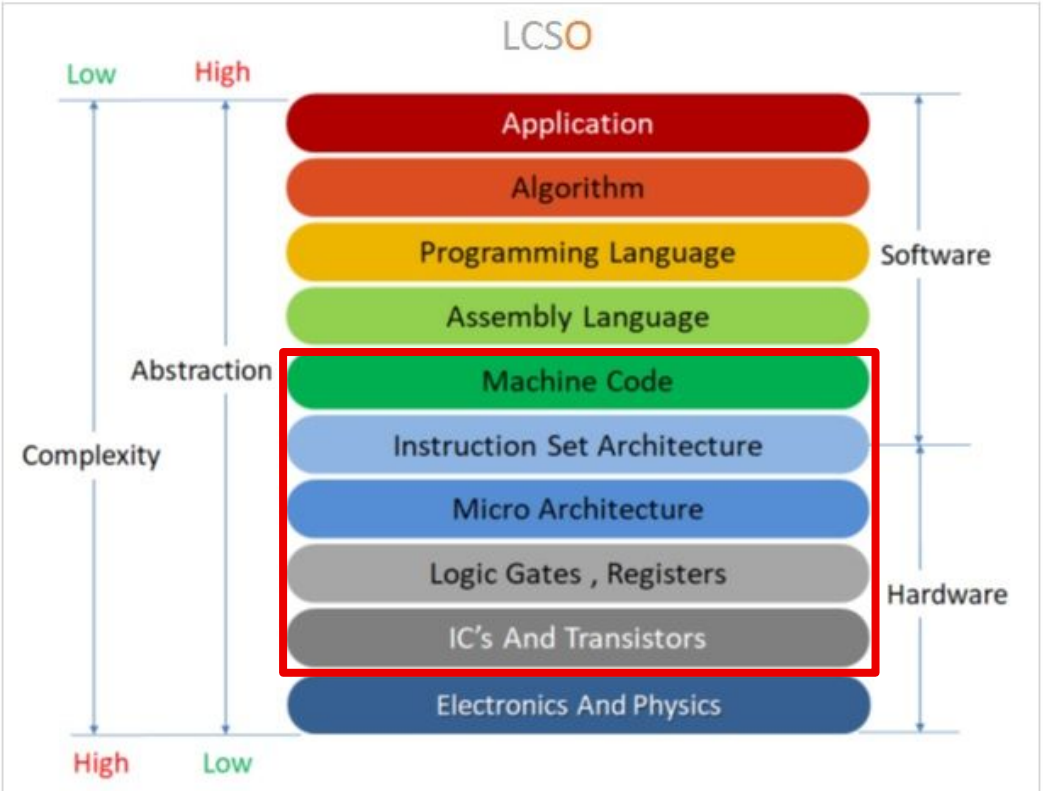




When do we use FPGA? ASICs?

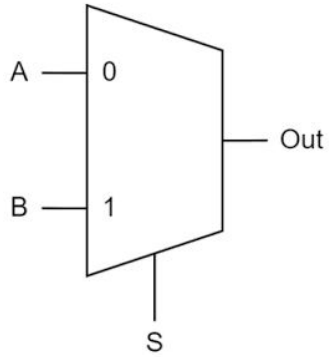


Why abstract? Why not just improve transistors?

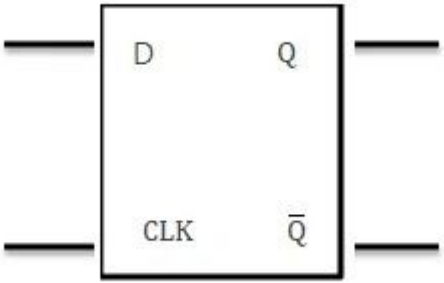


Logic Synthesis

```
always_comb begin
  if (in_valid) begin
    y = a;
  end else begin
    y = b;
  end
end
```




```
always_ff @(posedge clk) begin
  q = d;
end
```



Logic Synthesis (Gate to RTL)

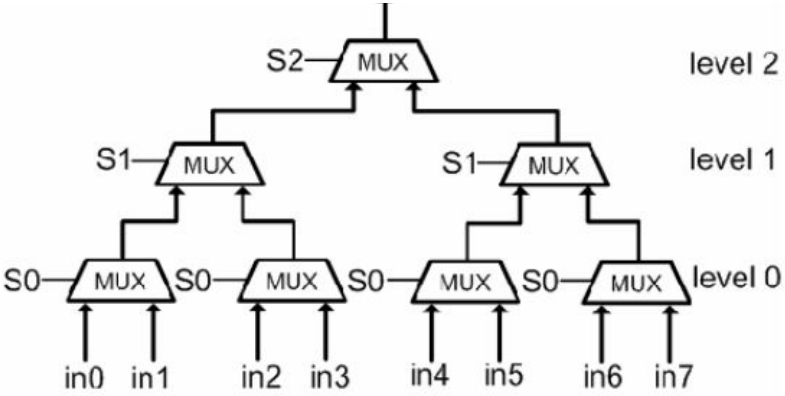
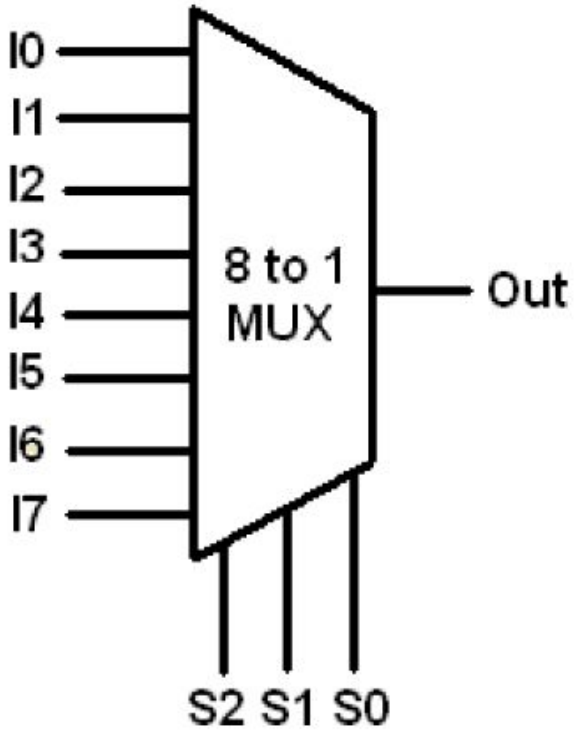


Figure 1. A traditional MUX tree.



Logic Synthesis (Gate-to-RTL, Sol.)

```
always_comb begin
  if (cond1) begin

  end else if (cond2) begin

  end else if (cond3) begin

  end else if ...

  end
end
```

Fig 1: RTL for Mux Tree

```
always_comb begin
  case (cond)
    2'b00: ...
    2'b01: ...
    2'b10: ...
    2'b11: ...
  endcase
end
```

Fig 2: RTL for Large Mux

Inferred Modules

What are inferred modules and what causes them? Examples?

- Flip-flop (generated via sensitivity list, i.e. @(posedge clk))
- Latch (adds state to combinational logic)
- Memory Elements (e.g. ROM, BRAM, URAM, in FPGA, ASICs are diff.)
- Adders (instantiated in ALUs)
- Clock Gating Elements (advanced, more rarely used)

Inferred modules are sometimes unintentional (the most common being latches); this can cause issues, like making STA harder, glitches, etc.

Latch Debugging

Which of the following infer a latch?

```
always_comb begin
  if (sel) begin
    y = x;
  end else if (enable) begin
    y = a;
  end
end
```

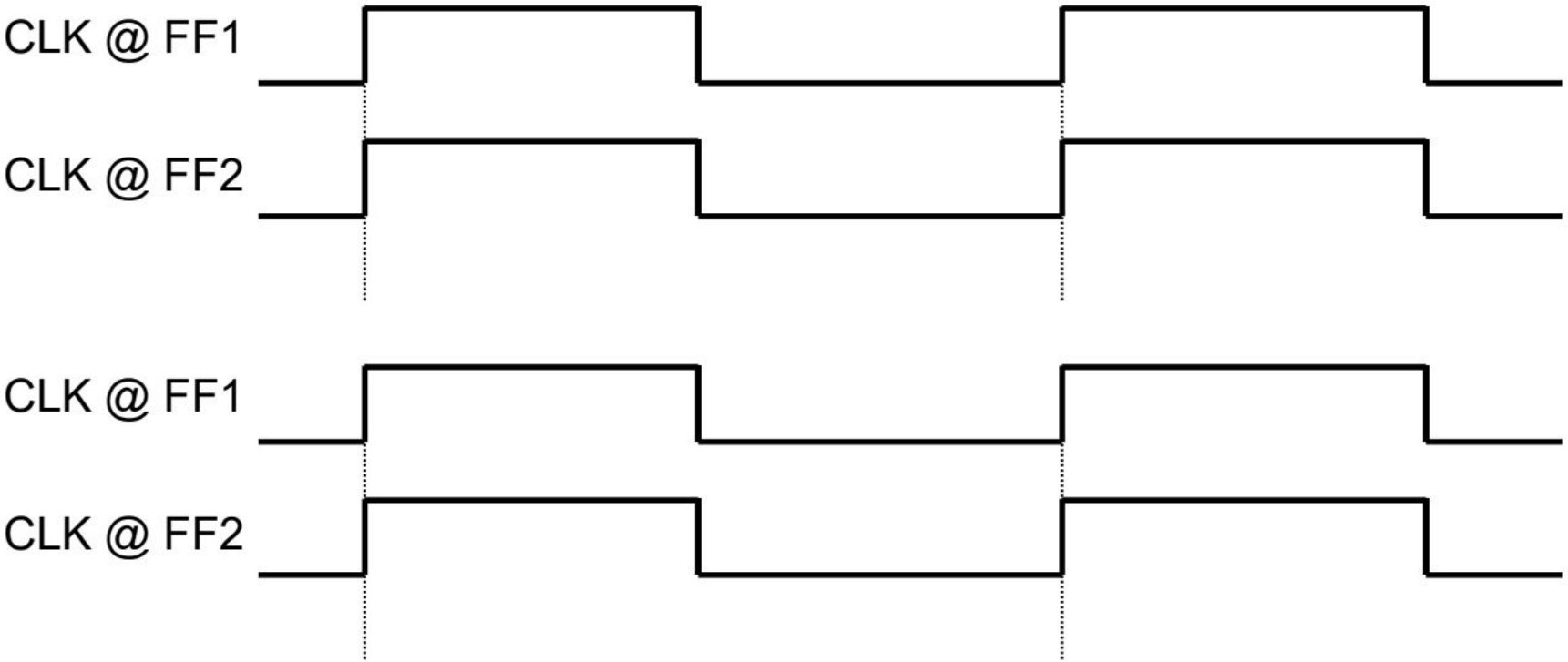
```
always_comb begin
  x = in;
  for (integer a = 1; a < 32; a *= 2) begin
    x ^= x >> a;
  end
end
```

```
always_comb begin
  if (en)
    a[7:4] = b;

  a[3:0] = c;
end
```

```
always_comb begin
  for (int i = 0; i < 4; i++) begin
    if (en[i])
      y[i] = a[i];
  end
end
```

Setup & Hold Violation (hand-drawn example)



Power, Performance, Area

- Power
  - How much power does our design expend/need?
  - Important concern in ASICs, low-power designs, etc.
- Performance
  - How well does our design perform?
  - Not always a single metric… some metrics are better than others!
- Area
  - How much hardware does our design take to build?
  - Varies based on process, also usually correlates directly with cost

What metric is the most important?

Static & Dynamic Power

Power can be categorized into two types: static and dynamic.

Static power is power expended when our design is not being operated. Static power is directly proportional to the area of our design - more transistors means more leakage. There are ways to reduce static power, at the architectural and transistor level.

Dynamic power is power expended when our design is operating - we want to keep this minimal for the most part as well, and there are power saving methods to do so.

In general, however, with higher performance and higher area, we'll also have higher power costs. Architects are still needed to perform the tradeoffs between PPA.

Performance

Performance is often determined by the ‘critical path’.

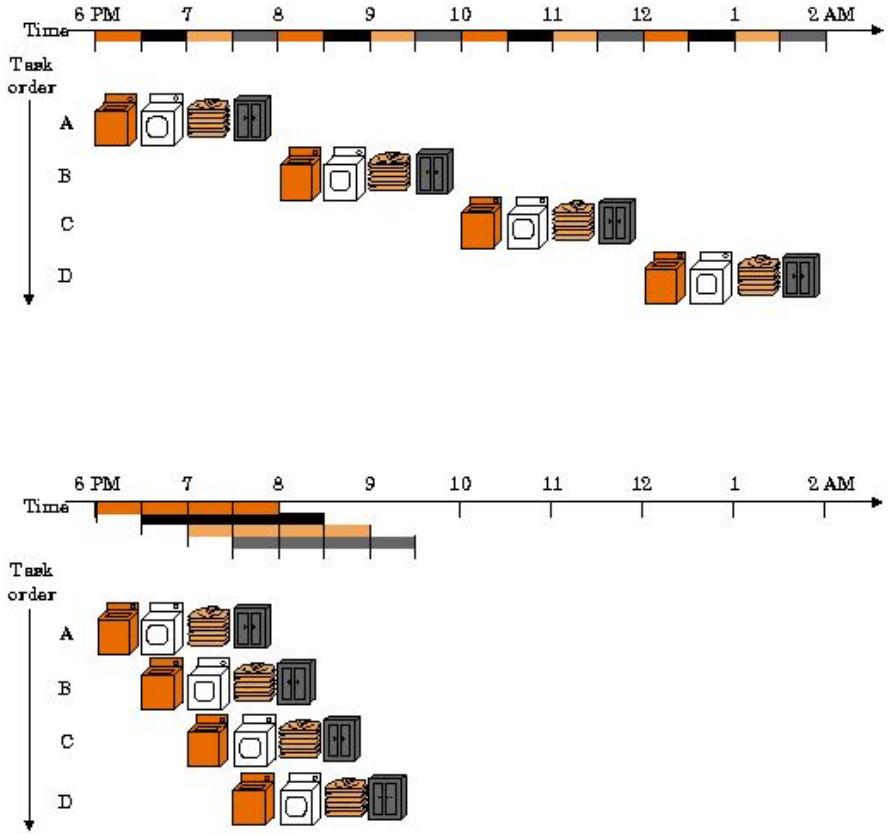
Critical Path: longest time between any two flip-flops within a circuit.

With a smaller critical path, our ‘period’ improves, so we can run our design at a higher clock frequency - this is good!

Techniques that reduce critical path:

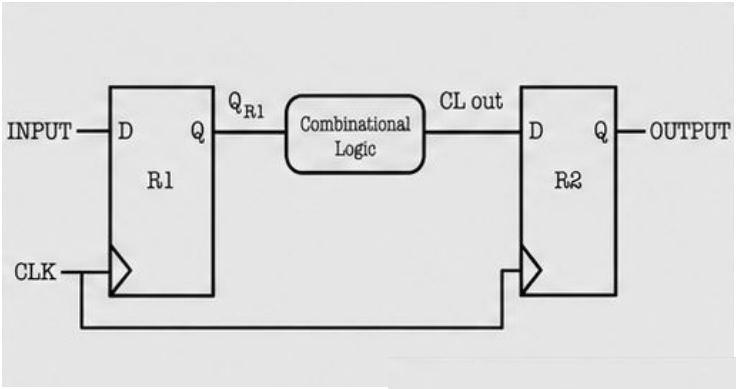
1. Pipelining
2. Clock Skewing (inserting buffers between consecutive flip-flops)
3. Time-borrowing (using latches instead of FFs to steal time between latches)
4. Logic Retiming (moving logic between other FFs)
5. Parallelism… (reducing comb. logic sequentialism and do more parallel work)

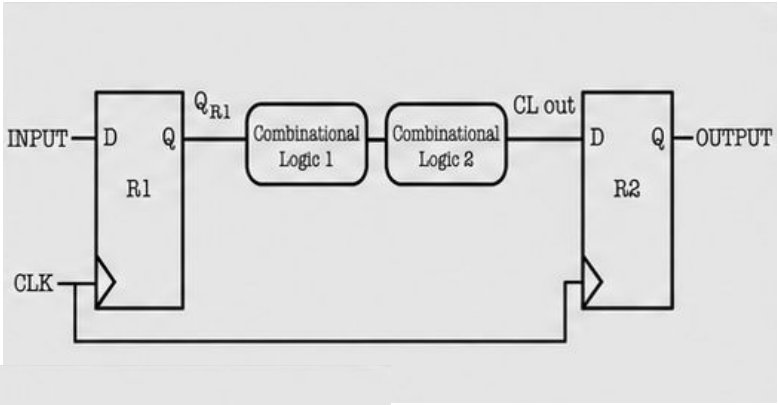
Pipelining

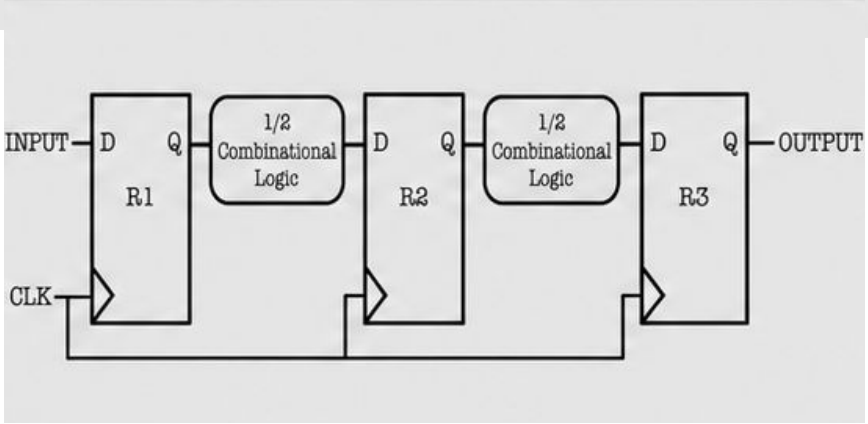

6 PM
7
8
9
10
11
12
1
2 AM
Time
Task order
A
B
C
D
6 PM
7
8
9
10
11
12
1
2 AM
Time
Task order
A
B
C
D

Critical Paths

The critical path is the longest combinational logic between any two FFs.







Area

Area is the indicator of our ‘hardware complexity’. Area is defined as our total physical silicon area, consumed by standard cells + macros, and resources include (but not limited to):

- Standard Cells
  - Combinational Logic
    - Gates, Muxes, Adders
  - Sequential Elements
    - Registers, FFs, Latches
- Memory Macros
  - SRAM, ROM, RFs
- Other Hard Macros
  - PLLs, Analog IPs, etc.

Attend the PD lecture to learn more :)

Finite-State Machines

A finite-state machine refers to a model that can be in any single state at any given time. The machine has a deterministic number of states, and additionally can be either ‘Mealy or Moore’.

FSM Logic Rule of Thumb:

- Use combinational logic to generate next state and outputs.
- Use clock edge to trigger state transition.
- Use parameters or enums to define state names/values.

Moore machines generate outputs based on the current state, and Mealy machines generate outputs based on the current state and inputs.

FSM Design Example

A gambling machine takes 10s and 20s. When 40 is put in, it begins the game, but if more than 40 dollars are put in, then it has to return the rest, and it can only return 10 dollar bills. All outputs must pulse for one cycle. If cancel goes high, it must return all bills it has taken so far, and pulse return_ten until all bills are returned, ignoring all inputs, bar reset.

```
module slot_machine (
        input logic clk, rst_n,
        input logic ten_in, twenty_in,
        input logic cancel,
        output logic start_slots,
        output logic return_ten
);
```

Design & Verification

It's also the designer's responsibility to verify their module! A module that blatantly does not work is a module that cannot be repaired. I recommend learning SVA; concurrent assertions are your best friend.

```
assert property (@(posedge clk) disable iff (!rst_n)
    (out_valid && out_ready) |-> (out_data != $past(out_data))
);

assert property (@(posedge clk) disable iff (!rst_n)
    (in_ready && !in_valid) |=> (in_ready)
);

assert property (@(posedge clk) disable iff (!rst_n)
    ((din) ##1 (din) ##1 (!din) ##1 (din)) |=> (detect)
);
```

Handshaking & Buffers

When designing modules, we want to ensure that modules satisfy behavioral requirements, without hard-coding in cycle-level behavior.

```
module handshake (
        input logic clk,

        input logic in_valid,
        output logic in_ready,

        output logic out_valid,
        input logic out_ready
);
```

Clock-Domain Crossing

When designing ASICs, there are many cases in which you need to send data between different clocks (e.g. producer-consumer behavior, communication protocols). We call this process ‘clock-domain crossing’.

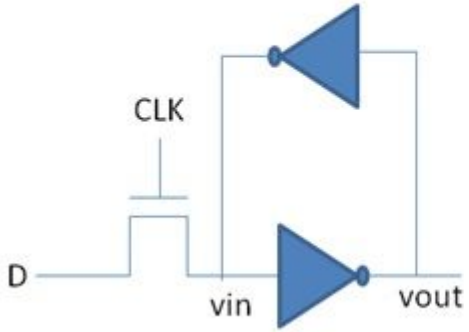
However, there are issues. What if Clock A and B are completely different speeds, and not similar?

If data at Clock A is produced 3x faster than Clock B can consume, data will be dropped. In the opposite direction, Clock B may consume data 3x faster, leading it to duplicate repetitive data and believe it is independent.

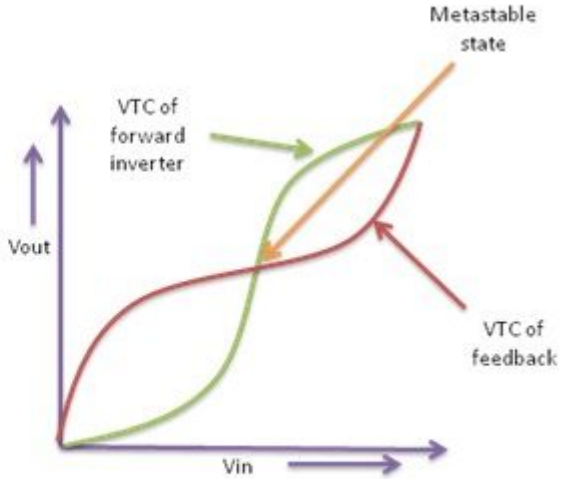
CDC schemes often use various techniques to ensure that each data item is transferred exactly once despite mismatched clock rates.

Metastability

When logic changes (e.g. output goes from 0 to 1), there is a period of time where the voltage is changing called ‘metastability’.

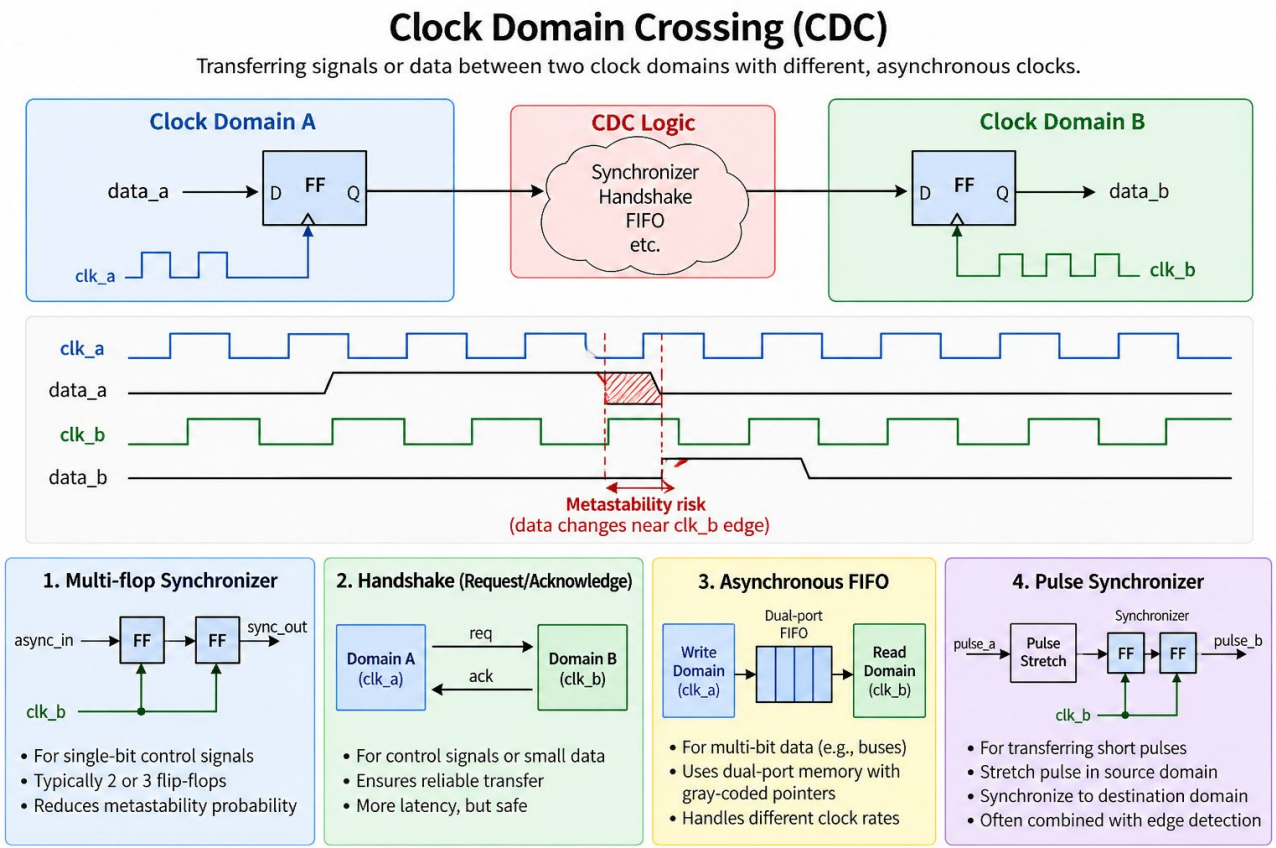


A simple D latch

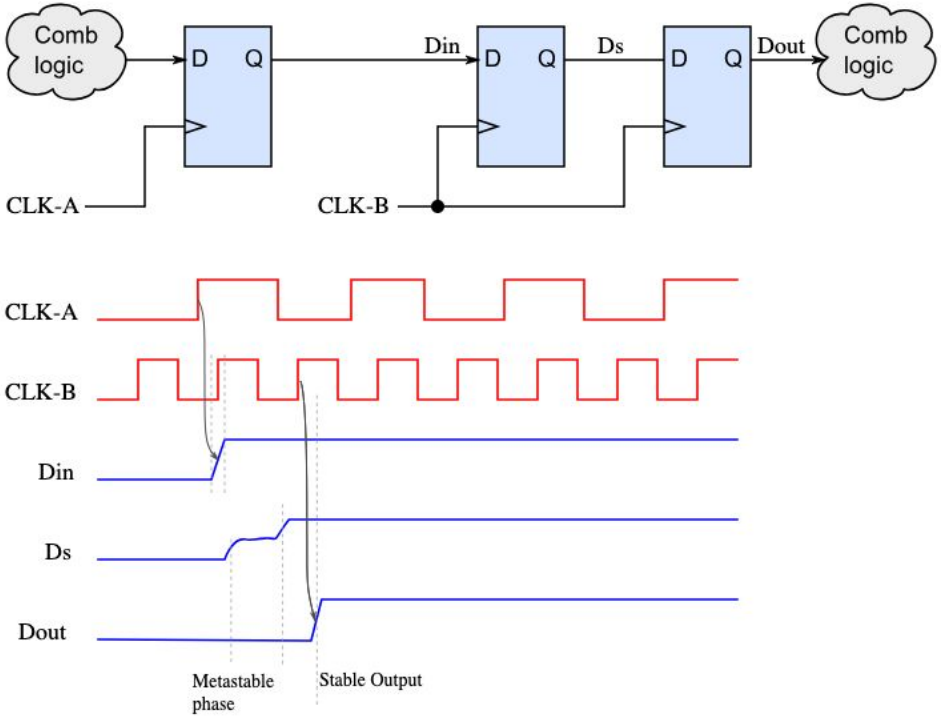


Plot showing transfer curves of the two inverters in D latch

Synchronizers



2-FF Synchronizers and MTBF



The Reliability Formula

$$R_{(t)} = e^{-(\lambda t)}$$

Where:

$R_{(t)}$ = reliability at a specific time period ***t***

e = base of natural logarithms (2.718281828)

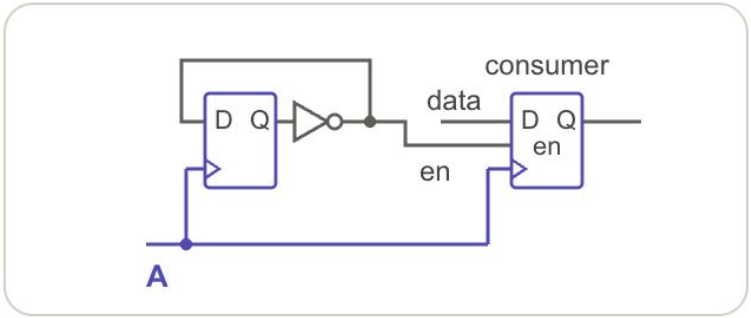
t = specific time period (or cycle)

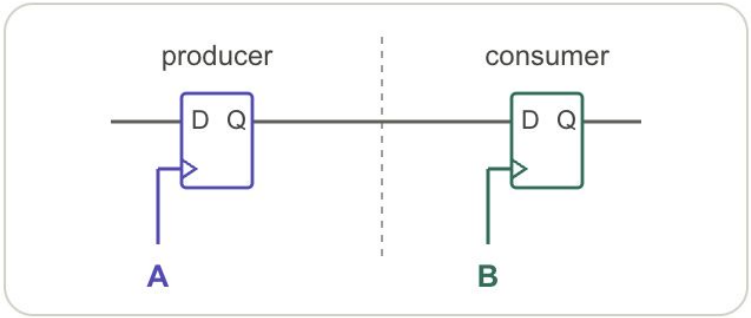
λ = failure rate = $\frac{1}{\text{MTBF}}$

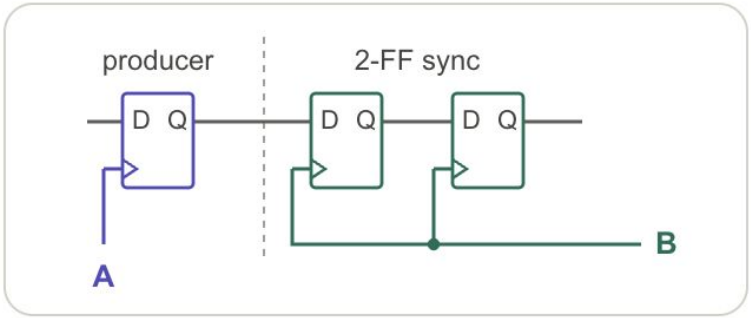
MTBF = Mean Time Between Failure

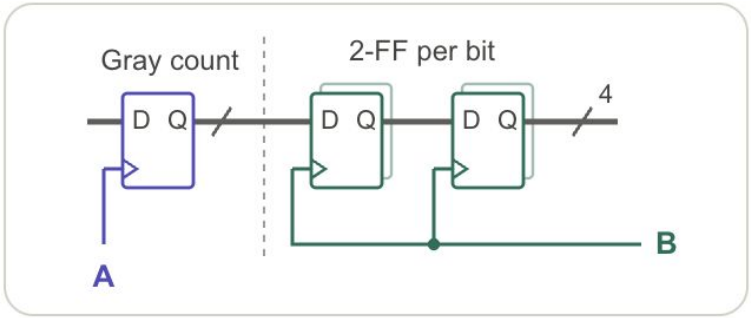
Example: Legal Synchronicity

Assume Clock A (producer) is 2x as fast as Clock B (consumer) with no phase shift. Which of the following are legal ways to synchronize data?









UCLA

Synchronous FIFO Design

Design a synchronous FIFO with support for handshaking. It must accept parameters of FIFO depth and data width. Outputs may be combinationally assigned (i.e., first-word fall-through FIFO).

```
module sync_fifo_w_handshaking #(
        parameter FIFO_DEPTH = 32,
        parameter DATA_W = 16
) (
        input logic clk,
        input logic rst_n,

        input logic in_valid,
        output logic in_ready,
        input logic [DATA_W-1:0] w_data,

        input logic out_ready,
        output logic out_valid,
        output logic [DATA_W-1:0] r_data
);
```

Computer Architecture

Arch, Lecture 1

Attendance and Team Creation Forms

Fall Capstone

You need to build an 8-bit ISA, and the underlying architecture that can execute the instructions you've promised in your ISA :)

Bonus points for more programmability (e.g. branching), PPA (we're still ironing out our evaluation metrics here), and overall creativity.

Requirements:

- Your ISA must allow some specific arithmetic operations.
- It must fit within a 1x1 TinyTapeout tile.
- Additionally, your capstone must have an SPI peripheral to communicate with off-chip microcontrollers for programmability.
- You can form teams of 4-5!

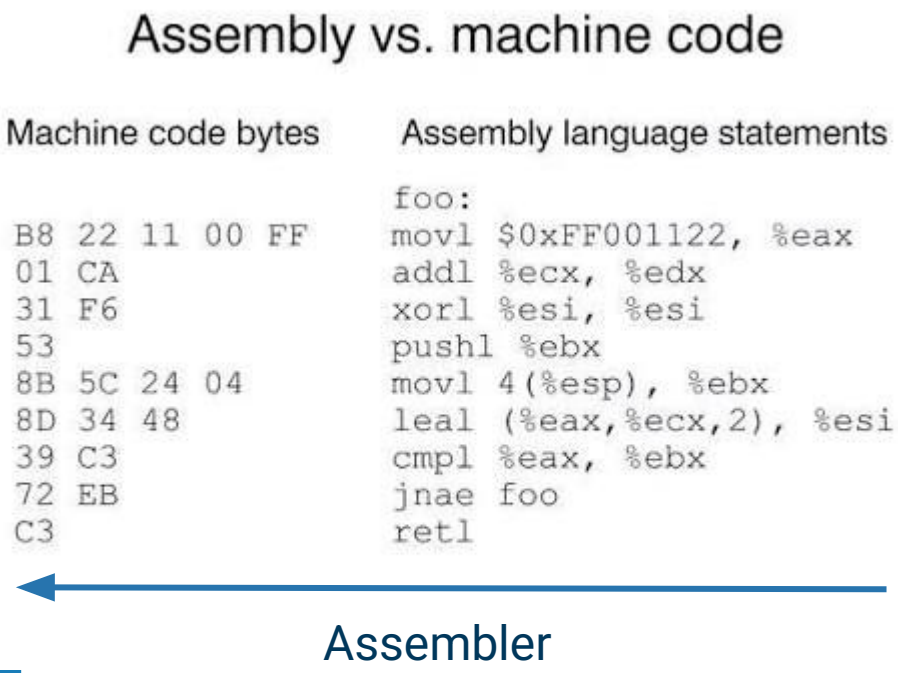
Fall Capstone

We recommend trying to stay ahead of Fall Capstone! It's a huge endeavor that took us a few weeks to implement over the summer (we were all asynchronous, which was tough).

To make your life easier, we recommend parallelizing your implementation of the RTL and verification – write your testbenches simultaneously as your teammates write RTL, etc.

If you need help with any specific part (e.g. verif, RTL, PD), reach out to the corresponding lead. **Please aim to stay on top of things!** Without Fall Capstone done, we cannot ensure you will be well prepared for the rate we will iterate in Winter Quarter, and you **will** fall behind.

Sequential Programmer Model



When a programmer writes code, they expect A to happen before B.

In HDLs (as we know) this is not always true :)

This is called the sequential program model. The programmer doesn't care about the computer below, as long as the code works!

What should a programmer be able to do?

As programmers, what do you want to be able to express?

- loops (for, while, etc.)
- if-else statements
- arithmetic operations
- storing and loading data (malloc)
- calling functions
- different data types
- vector operations
  - perform operations over two arrays
- multithreading (parallelism)
  - synchronization
- system calls (file-system, etc.)

ISA (Instruction Set Architecture)

RV32I Base Integer Instructions

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2	
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2	
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2	
or	OR	R	0110011	0x6	0x00	rd = rs1 | rs2	
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2	
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2	
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2	
sra	Shift Right Arith*	R	0110011	0x5	0x20	rd = rs1 >> rs2	msb-extends
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0	
sltu	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0	zero-extends
addi	ADD Immediate	I	0010011	0x0		rd = rs1 + imm	
xori	XOR Immediate	I	0010011	0x4		rd = rs1 ^ imm	
ori	OR Immediate	I	0010011	0x6		rd = rs1 | imm	
andi	AND Immediate	I	0010011	0x7		rd = rs1 & imm	
slli	Shift Left Logical Imm	I	0010011	0x1	imm[5:11]=0x00	rd = rs1 << imm[0:4]	
srli	Shift Right Logical Imm	I	0010011	0x5	imm[5:11]=0x00	rd = rs1 >> imm[0:4]	
srai	Shift Right Arith Imm	I	0010011	0x5	imm[5:11]=0x20	rd = rs1 >> imm[0:4]	msb-extends
slti	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm)?1:0	
sltiu	Set Less Than Imm (U)	I	0010011	0x3		rd = (rs1 < imm)?1:0	zero-extends
lb	Load Byte	I	0000011	0x0		rd = M[rs1+imm][0:7]	
lh	Load Half	I	0000011	0x1		rd = M[rs1+imm][0:15]	
lw	Load Word	I	0000011	0x2		rd = M[rs1+imm][0:31]	
lbu	Load Byte (U)	I	0000011	0x4		rd = M[rs1+imm][0:7]	zero-extends
lhu	Load Half (U)	I	0000011	0x5		rd = M[rs1+imm][0:15]	zero-extends
sb	Store Byte	S	0100011	0x0		M[rs1+imm][0:7] = rs2[0:7]	
sh	Store Half	S	0100011	0x1		M[rs1+imm][0:15] = rs2[0:15]	
sw	Store Word	S	0100011	0x2		M[rs1+imm][0:31] = rs2[0:31]	
beq	Branch ==	B	1100011	0x0		if(rs1 == rs2) PC += imm	
bne	Branch !=	B	1100011	0x1		if(rs1 != rs2) PC += imm	
blt	Branch <	B	1100011	0x4		if(rs1 < rs2) PC += imm	
bge	Branch ≥	B	1100011	0x5		if(rs1 >= rs2) PC += imm	
bltu	Branch < (U)	B	1100011	0x6		if(rs1 < rs2) PC += imm	zero-extends
bgeu	Branch ≥ (U)	B	1100011	0x7		if(rs1 >= rs2) PC += imm	zero-extends

Promises that are made to the programmer about what the core can do…

This ISA promises integer math, memory operations, branching.

Your underlying hardware just needs to do this! The ISA makes no promises about how the hardware does it, just what the programmer should see.

Instruction Field Encoding

RISC-V Instruction Set

Core Instruction Formats

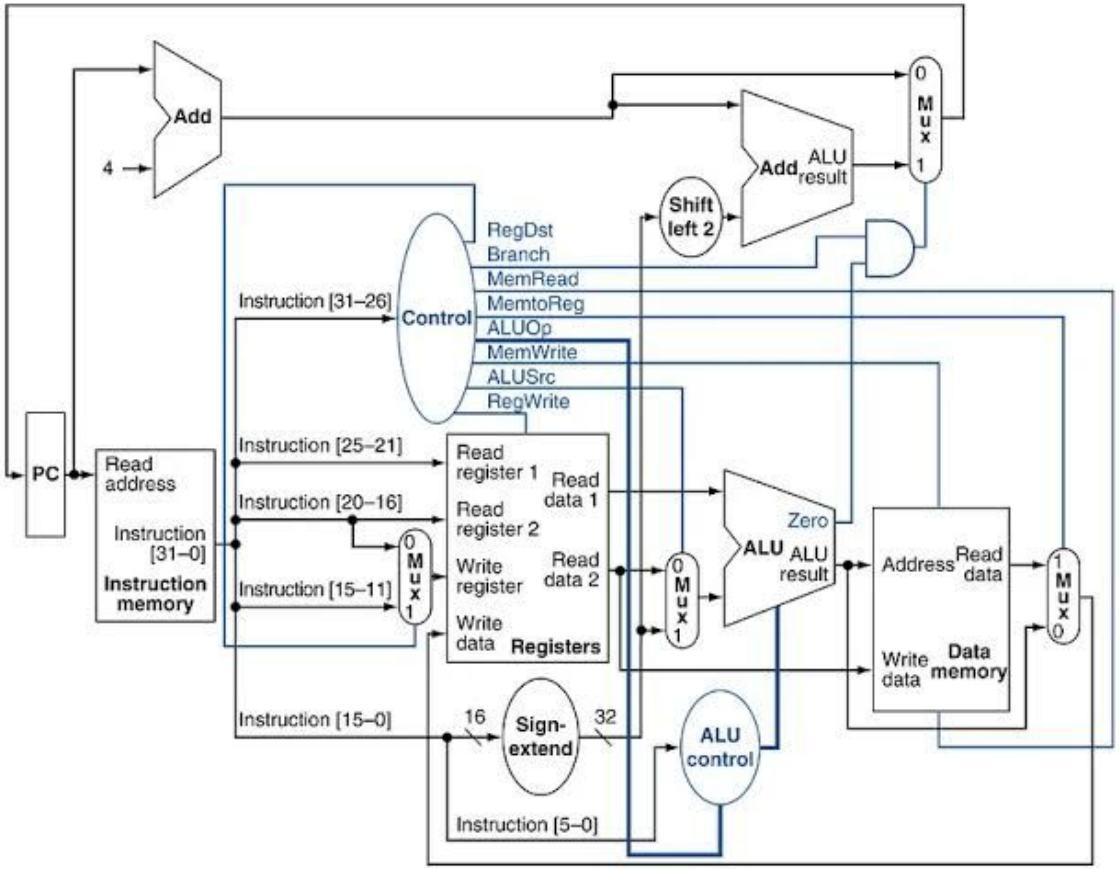
31 27	26 25	24 20	19 15	14 12	11 7	6 0	
funct7		rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]			rs1	funct3	rd	opcode	I-type
imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode	S-type
imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode	B-type
imm[31:12]					rd	opcode	U-type
imm[20|10:1|11|19:12]					rd	opcode	J-type

Capstone, Step 1: ISA Development

1. What features do you want to express?
   a. Branching, Load/Store, etc.
   b. For reference, our implementation expressed one data register, and didn't include branching.
2. How can we encode registers and other fields into our instruction?
   a. We implemented only rd and rs1, so all operations looked like:

rd = rd [op] rs1

Single Cycle RISC-V Datapath



Capstone, Step 2: Microarchitectural Impl.

1. Based on the features you want to implement from the ISA, generate your block diagram or edit an existing one!
    a. This ensures you and your teammates are on the same page.
2. Ensure your instruction encoding is optimal to minimize hardware complexity, otherwise restructure it.
3. Devise the list of modules you need to implement for the datapath, and write the interfaces for them (this is for ease of division).
4. Implement the datapath and test functional correctness.
    a. If not correct, reimplement and find the bugs.

Capstone, Step 3: SPI Slave

1. Devise the FSM protocol you want to have surrounding your datapath.
   a. i.e., how do you load instructions and tell the CPU when to operate?
2. If you have no idea how to write an SPI slave, I've written a slave that will be hyperlinked in the capstone if you need it, but we recommend you write your own to learn the fundamentals.
3. Surround your SPI slave and datapath with the FSM you implemented.

If this seems vague or confusing to you, we do recommend using AI – building a SPI slave is typically not an important or difficult task, and many companies maintain a standard library. **Please, however, aim to write the FSM yourself!**

Capstone, Step 4: Full Verification

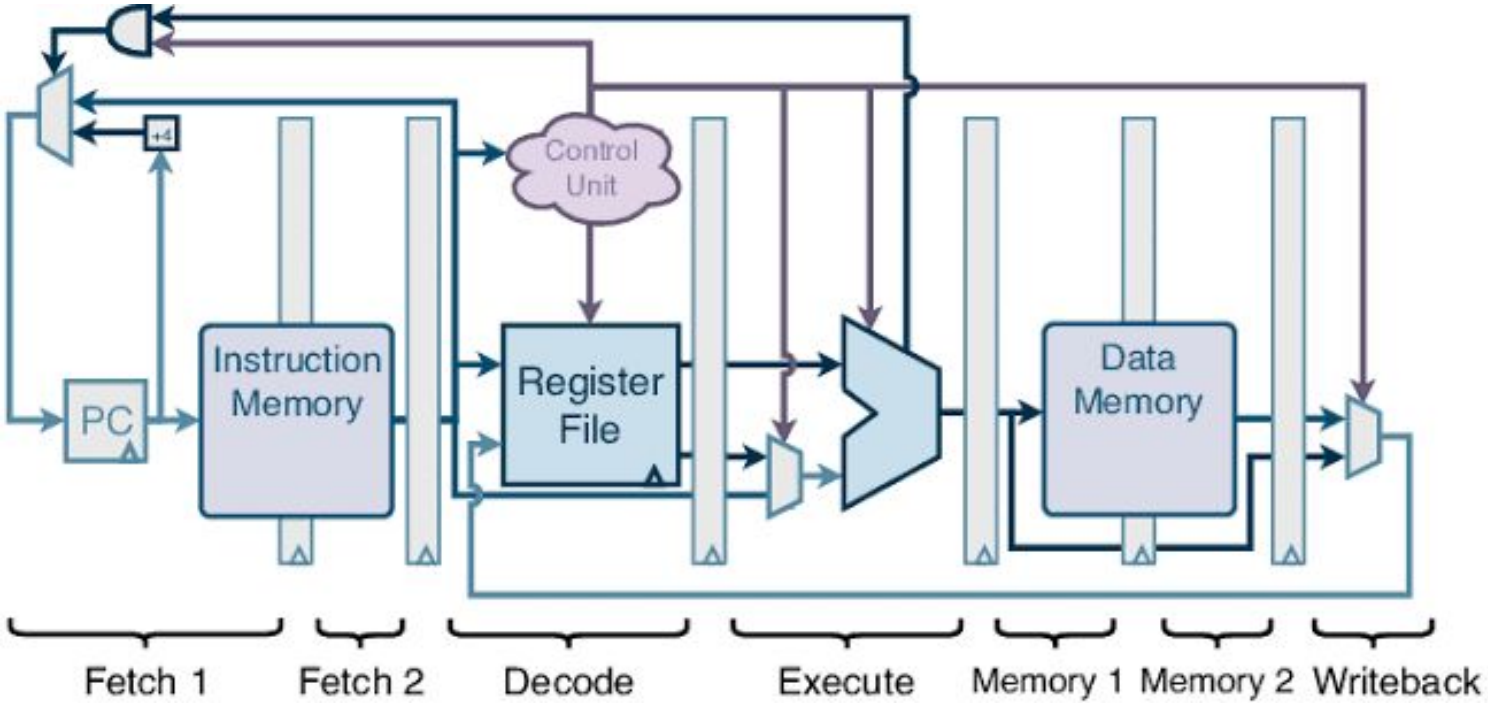
1. Individually verify your SPI Slave. Your datapath should already be verified from step 2.
2. Now, with your FSM wrapper integrating the SPI slave and the datapath, pass inputs asynchronously clocked into the SPI slave (similar to what a microcontroller would do).
3. Test the full datapath by reusing some of the sample programs, but ensure you extract the outputs via SPI slave as well (you can use the internal register state for debugging, but for true correctness, outputs must be extracted via slave as well).

Capstone, Step 5: Timing, Area Checks, etc.

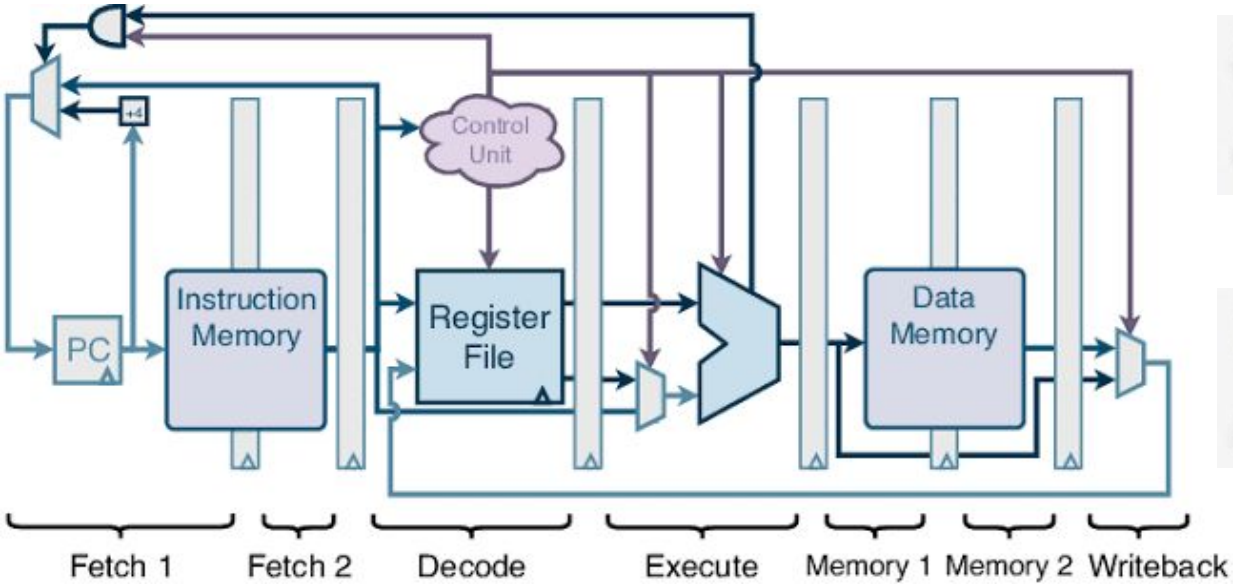
1. Run your full top-level module through the LibreLane flow on GitHub.
2. Read the STA and reports generated by the flow, and find your WNS (which should be positive).
3. Compute your max clock frequency, although your minimum should be 50 MHZ.
4. If you are unhappy with your timing or area, iterate and redo parts of your datapath, ISA, etc.
    a. No one ever said architecture and design was easy :)

If you feel happy and complete with your design, you're done! You've just completed the full ISA to GDSII process of an 8-bit CPU – a remarkable achievement for only a few weeks.

Pipelining the Datapath



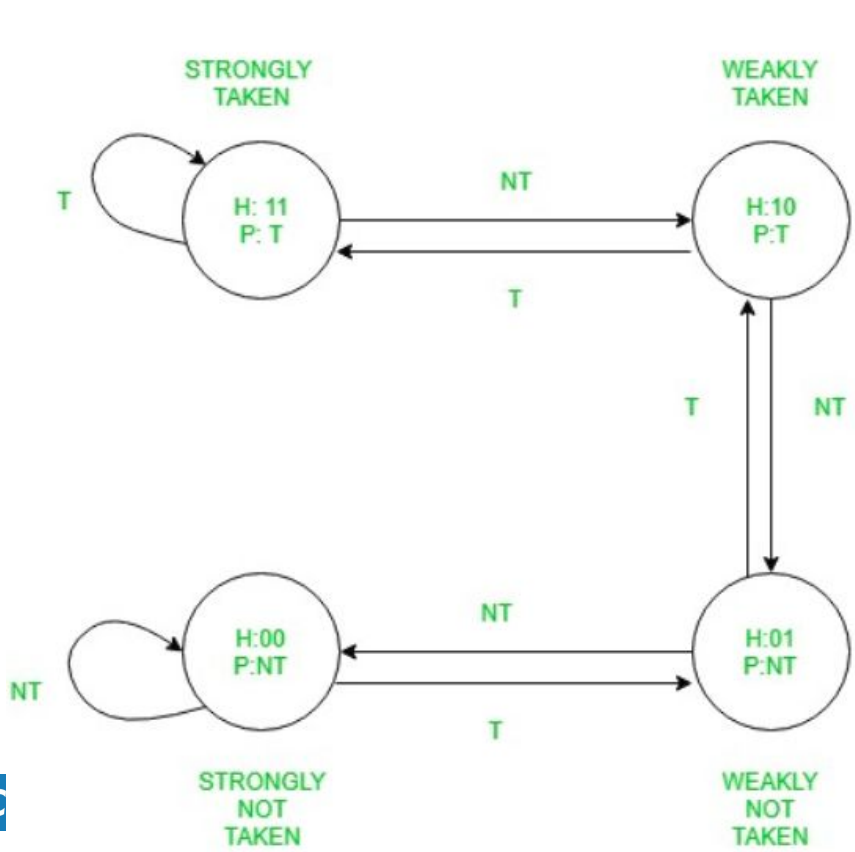
Forwarding & Load-Use Hazards

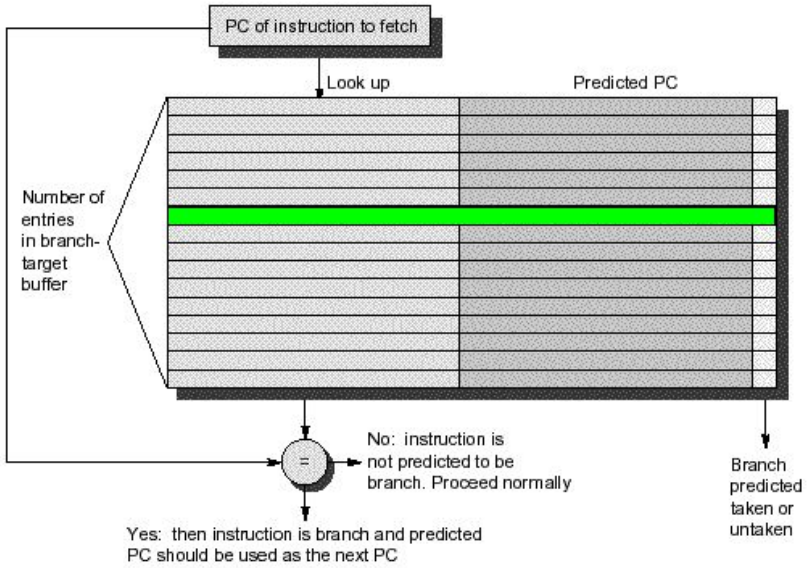



```
add   x7, x4, x5
add   x8, x7, x6
```

```
lw    x5, 0(x2)
add   x6, x3, x5
```

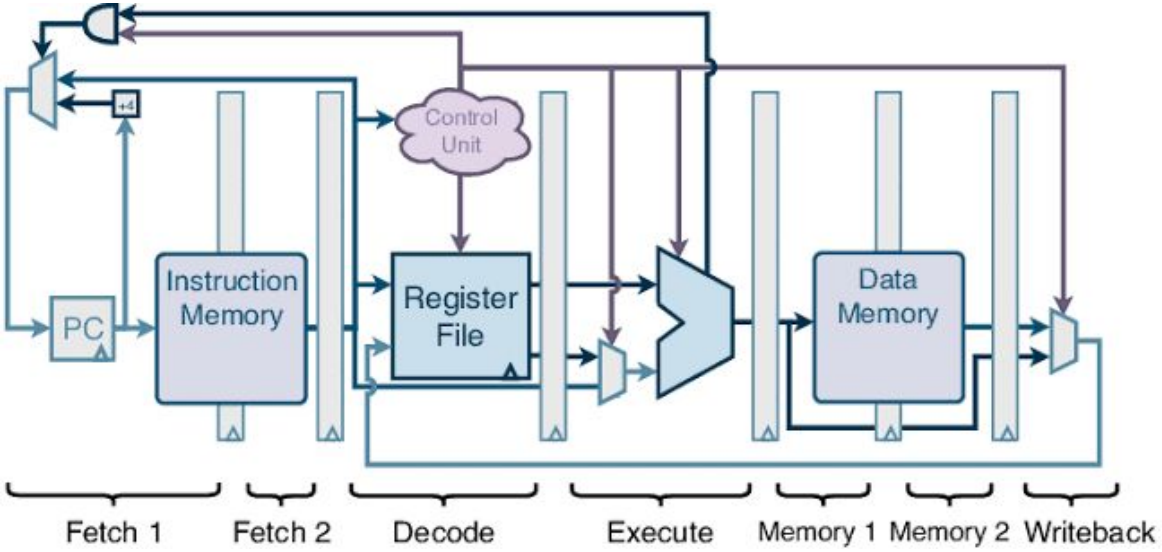
Branch Speculation (BHT, BTB)





Branch Misspeculation

Modern branch predictors have hit 95-97% accuracy on the majority of workloads! What happens if it's not right?



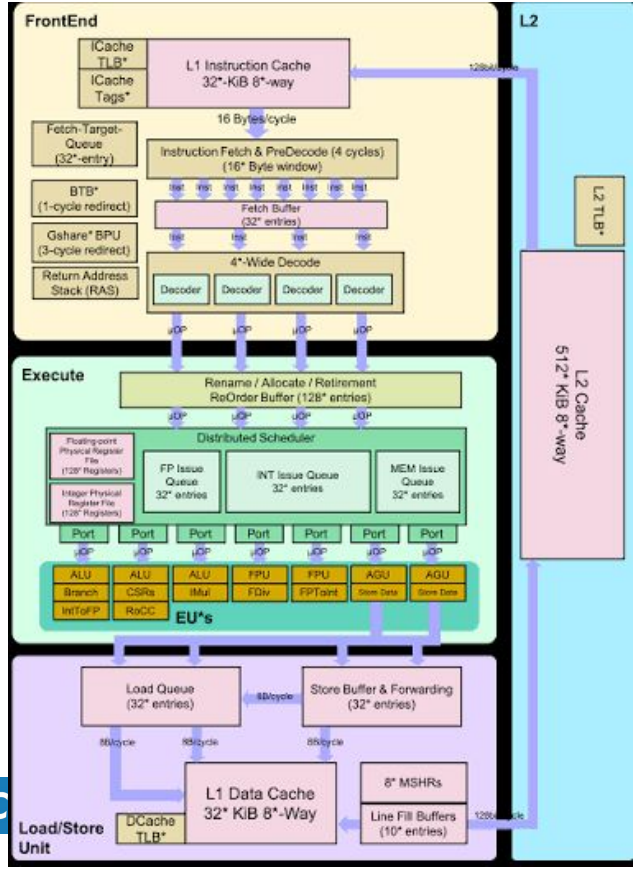
Speculative Processors

Modern processors perform many predictions, including the following:

- Branch Prediction
- Memory Disambiguation (i.e. out-of-order load-store speculation)
- Hardware Prefetching
- Cache Way Prediction
- Memory Dependence Prediction
- DRAM Row-Buffer Hit Prediction
- Value Prediction
- Thread-Level Speculation (Run-ahead Threads)

There are even more, depending on the processor!

O3, Superscalar



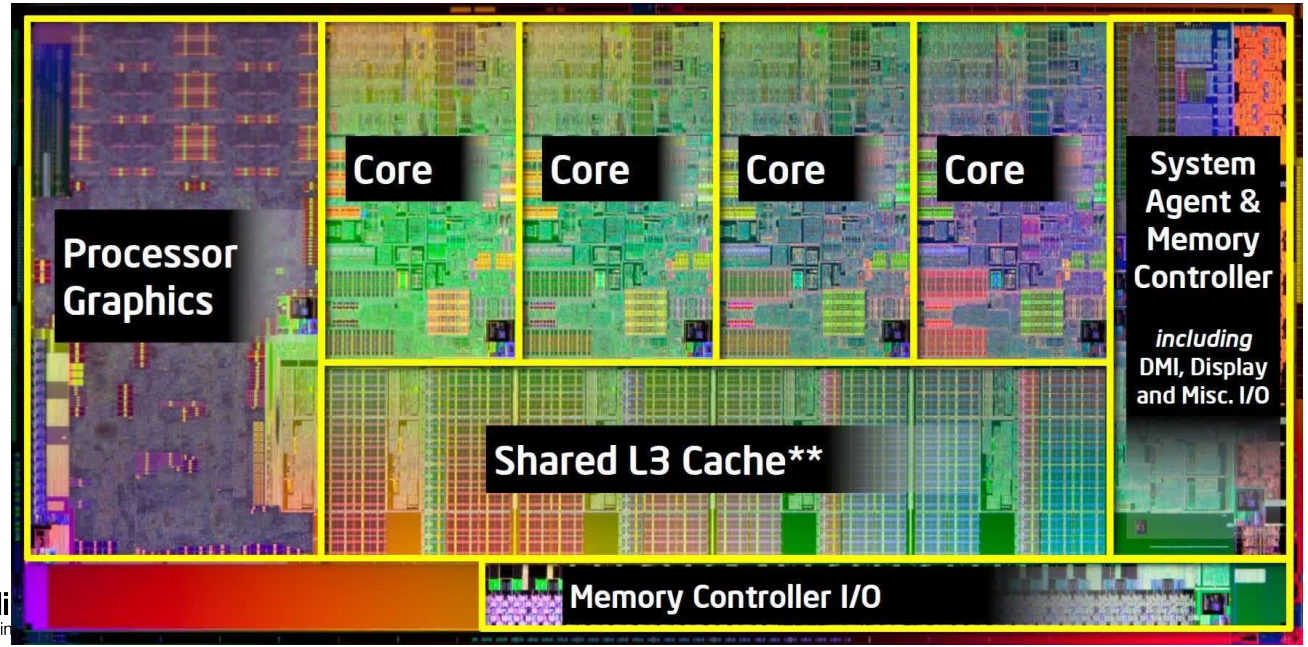
Out-of-order (O3): Instructions no longer have to be executed in sequential order. There now exists an ‘architectural’ state and a ‘physical’ state.

Superscalar: Multiple instructions can execute at once (i.e., you may have multiple ALUs). Sequential behavior is still typically followed, unless the core is explicitly out-of-order.

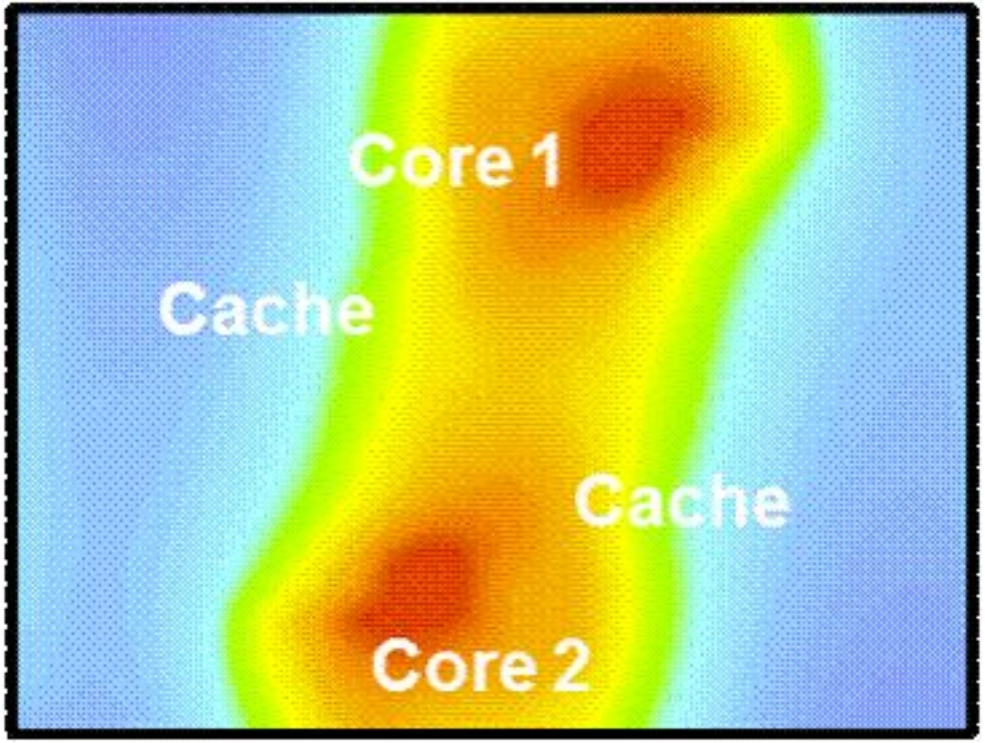
Variants exist that use superscalar, but not out-of-order (e.g. VLIW – read about Intel Itanium for why you’ve never heard about it).

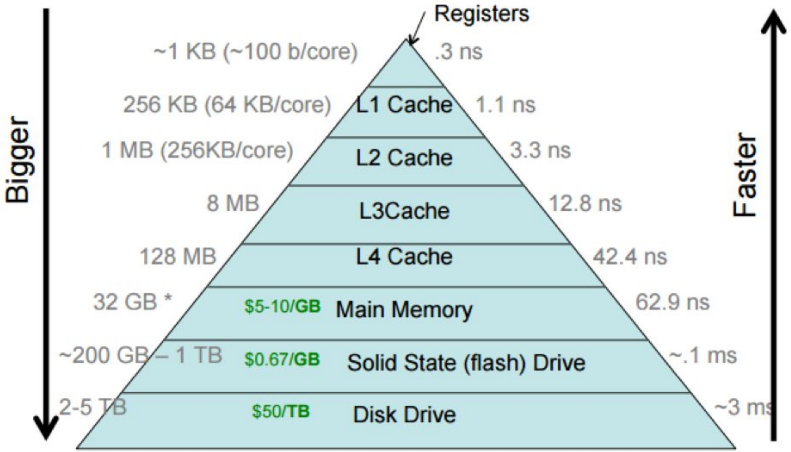
Multicore CPUs

Another layer of parallelism above O3 – improving the sizes of O3 structures has diminishing returns. Why not build more cores on chip, then let them share structures too (e.g. L3)? Coherency and consistency!



Processor Memories





Caching Fundamentals

Caches are good because they utilize two techniques:

- Spatial Locality – if you access data[0], you access data [1] too likely
- Temporary Locality – if you access data[0], you'll probably reuse it

Okay, but why not one large cache? Why have hierarchy?

- Speed of Access – with one large cache, all data is relatively slowly accessed, since it's so big (smaller caches output faster!)
- Cache Policies and Types – based on cache size, some cache types are better (for more cache reading, Primer on Caches)
- Individual Cache Optimizations (e.g. drowsy caching on L3, cuckoo hashing, memory type optimization, like eDRAM)

Learning Computer Architecture

Digital Tapeout will require a strong understanding of architecture; we can try, but we can't teach you everything (in fact, we have one lecture to teach this quarter to teach you weeks of M116C!) :(

If you're interested in learning more, we recommend M116C heavily, as well as 251A – these classes will provide enough of a foundation in computer architecture to pass any RTL interview for architecture companies.

You should also try to do some learning on your own! We recommend Nader's M116C lectures, primarily on out-of-order cores (we may develop them for the SoC), as well as cache design, etc.

ICG, PG, DVFS, Clocks

ASIC, Lecture 2

CDC Review

-

Dynamic Power

The largest power cost on-chip comes from the hierarchical clock-tree, as it switches every cycle. The clock can cost anywhere from 30-50% of total dynamic power consumption on chip.

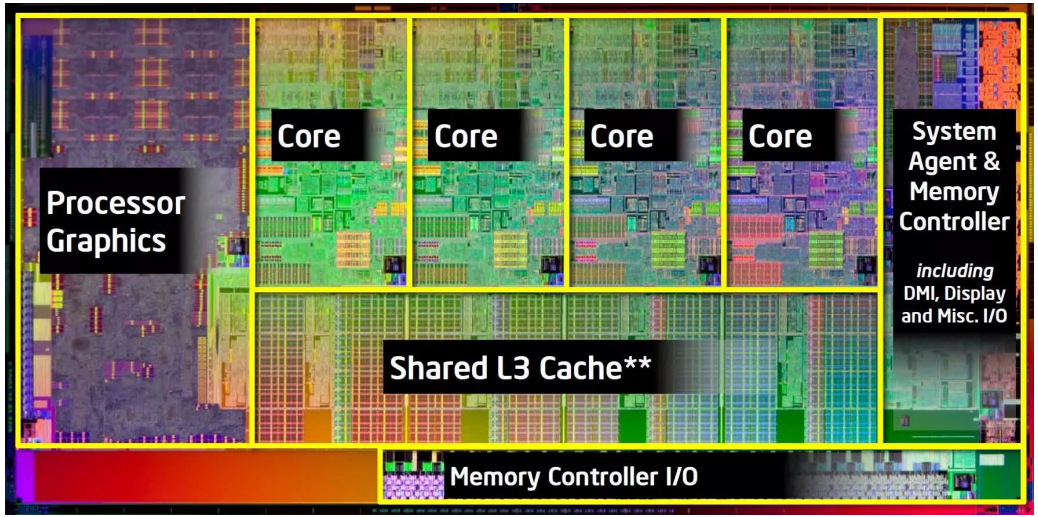
Solutions:

- Clock Gating
- Supply-Voltage Reduction
- Asynchronous Chips
- Clock-tree Optimization

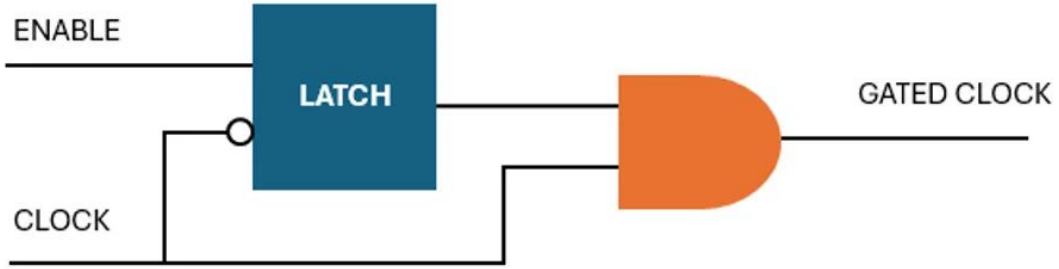
Clock Gating

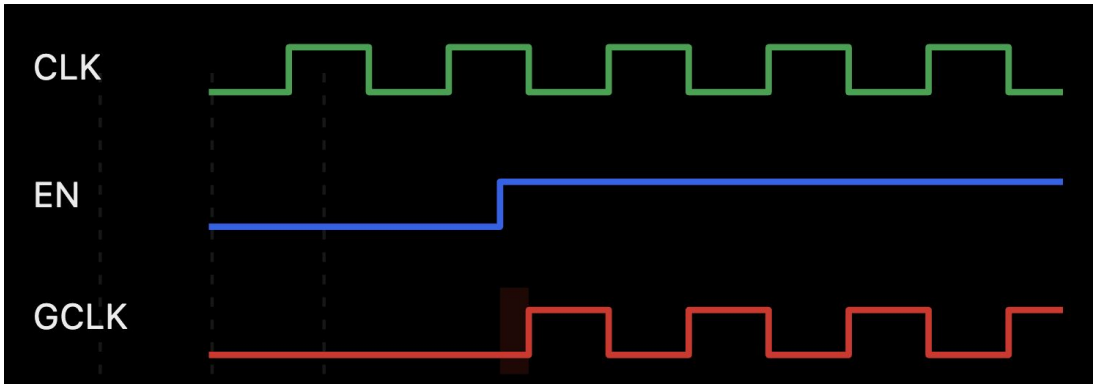
Imagine a multicore processor, but only one core is being used. The clock is still routed to the three other cores on-chip, which expends power.

We need a power management unit to tell the other cores to not switch the clock every cycle.



Integrated Clock-Gating Cells





Why not use an FF? Why use a latch?

Additionally, how should enable be handled? What hardware data structures can we use to generate enable for various locations on the chip?

Static Power

Static power is the cost of transistors idling, and leaking current even when they're turned off. Your gates not only cost dynamic power to perform logic, but cost power statically while existing!

Solutions:

- Power Gating
- Dynamic Supply Voltage Reduction
- Transistor Stacking
- Sizing Transistors
- Reverse Body Bias/Transistor Threshold Increase

Power Gating

-

Isolation Cells

-

Retention Latches

-

Dynamic Voltage-Frequency Scaling (DVFS)

-